

Aim: To study Support Vector Machine.

1.Import Libraries.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import micropip
await micropip.install(['seaborn'])
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix
```

2.Loading the Iris dataset.

CODE:

```
iris = load_iris()
X = pd.DataFrame(iris.data, columns = iris.feature_names)
y = pd.Series(iris.target, name = 'target')
X.head()
```

OUTPUT:

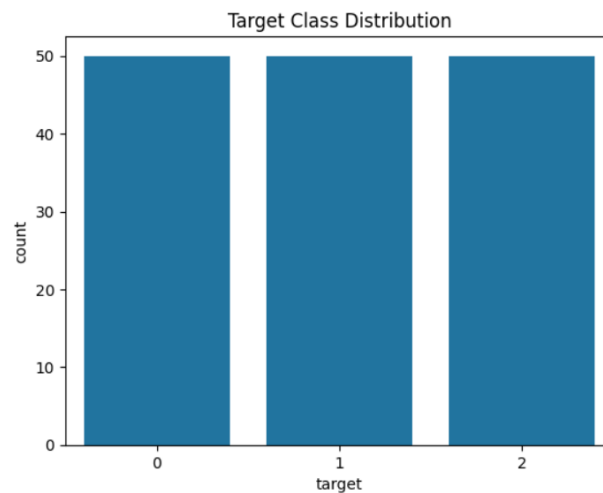
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

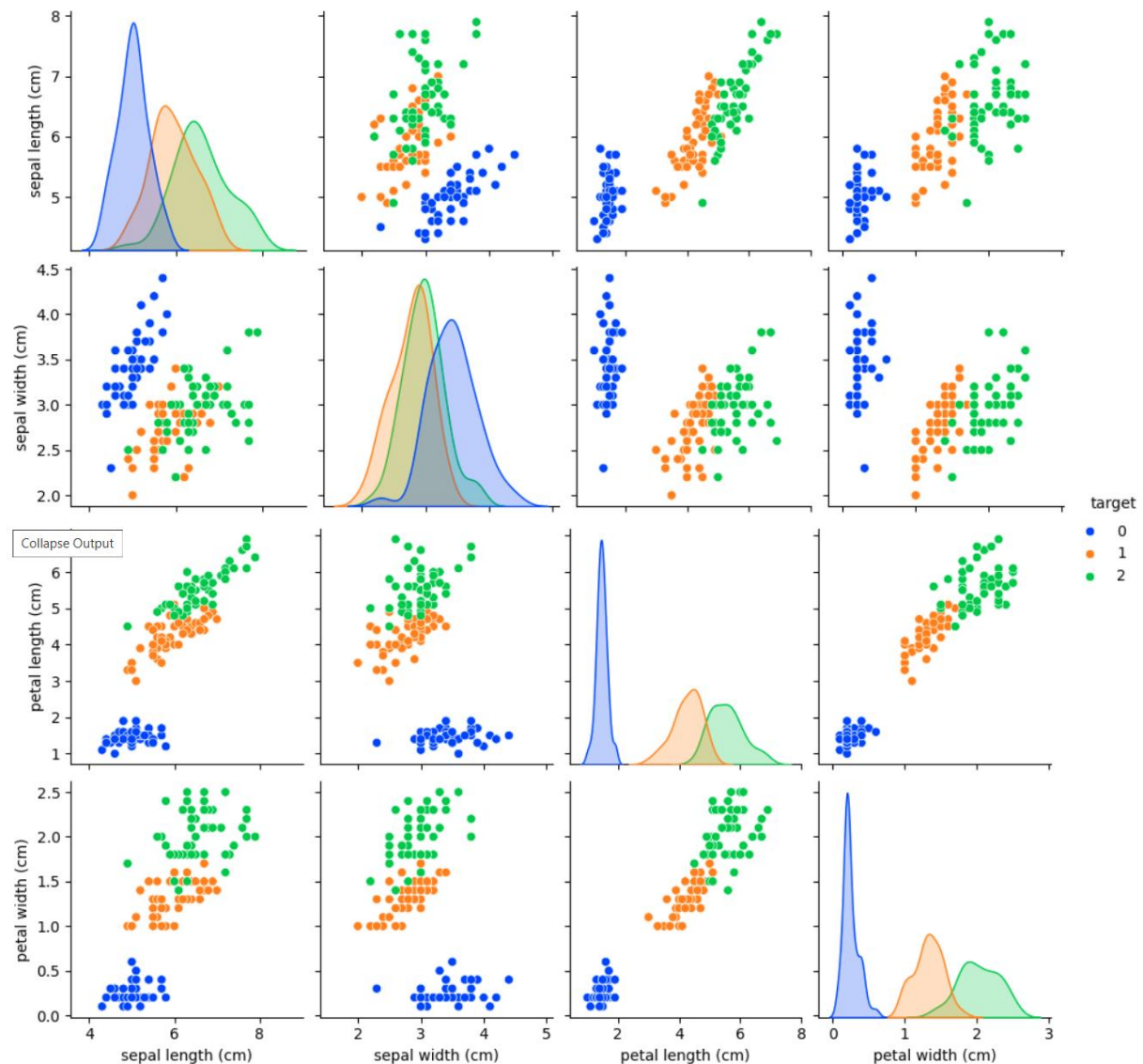
3.Combining the feature and target column and plotting the graph of it.

CODE:

```
sns.countplot(x=y)
plt.title("Target Class Distribution")
plt.show()
sns.pairplot(pd.concat([X,y], axis = 1), hue = 'target', palette = 'bright')
plt.show()
```

OUTPUT:





4.Training and testing the dataset.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state = 42, stratify = y)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

5.Getting the SVC model.

CODE:

```
svm_clf = SVC(kernel = 'linear', C = 1)
svm_clf.fit(X_train_scaled, y_train)
```

OUTPUT:

SVC

SVC(C=1, kernel='linear')

6.Displayig the classification and confusion matrix of the model.**CODE:**

```

y_pred = svm_clf.predict(X_test_scaled)
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

OUTPUT:

Confusion Matrix:

```

[[15  0  0]
 [ 0 14  1]
 [ 0  3 12]]

```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	15
1	0.82	0.93	0.88	15
2	0.92	0.80	0.86	15
accuracy			0.91	45
macro avg	0.92	0.91	0.91	45
weighted avg	0.92	0.91	0.91	45

7.Displaying the RBF i.e Radial Basis Functions classification report.**CODE**

```

svm_rbf = SVC(kernel = 'rbf', C = 1, gamma = 'scale')
svm_rbf.fit(X_train_scaled, y_train)
y_pred_rbf = svm_rbf.predict(X_test_scaled)
print("RBF Kernel Classification Report:")
print(classification_report(y_test, y_pred_rbf))

```

OUTPUT:

RBF Kernel Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	15
1	0.88	0.93	0.90	15
2	0.93	0.87	0.90	15
accuracy			0.93	45
macro avg	0.93	0.93	0.93	45
weighted avg	0.93	0.93	0.93	45

8. Generating the decision boundary of the model.

CODE:

```
X_vis = X.iloc[:, [0,2]].values
y_vis = y.values

X_train_vis, X_test_vis, y_train_vis, y_test_vis = train_test_split(X_vis, y_vis, test_size = 0.3, random_state = 42)
scaler_vis = StandardScaler()
X_train_vis_scaled = scaler_vis.fit_transform(X_train_vis)
X_test_vis_scaled = scaler_vis.transform(X_test_vis)

svm_vis = SVC(kernel = 'linear', C = 1)
svm_vis.fit(X_train_vis_scaled, y_train_vis)

import micropip
await micropip.install(['mlxtend'])
from mlxtend.plotting import plot_decision_regions

plt.figure(figsize = (8,6))
plot_decision_regions(X_test_vis_scaled, y_test_vis, clf = svm_vis, legend = 2)
plt.title("SVM Decision Boundary (Linear Kernel)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

OUTPUT:

